



Análisis y desarrollo de software.

ID:3145636



www.sena.edu.co



Instruct

or
Ing. Diego Fernando Calderon Silva
Correo: dfcalderon@sena.edu.co

JavaScr ipt

Objetos en JavaScript

En JavaScript, los objetos son estructuras de datos que permiten almacenar colecciones de pares clave-valor. Las claves son cadenas (o símbolos) y los valores pueden ser cualquier tipo de datos, incluidos otros objetos, funciones, arrays, etc. Los objetos se utilizan ampliamente para representar datos complejos y gestionar la estructura de las aplicaciones.

Creación de un objeto.



Puedes crear un objeto de varias maneras:

Literal de objeto:

```
let persona = {  
  nombre: "Juan",  
  edad: 30,  
  esEmpleado: true  
};
```

Usando new Object():

```
let persona = new Object();  
persona.nombre = "Juan";  
persona.edad = 30;  
persona.esEmpleado = true;
```

Creación de un objeto.



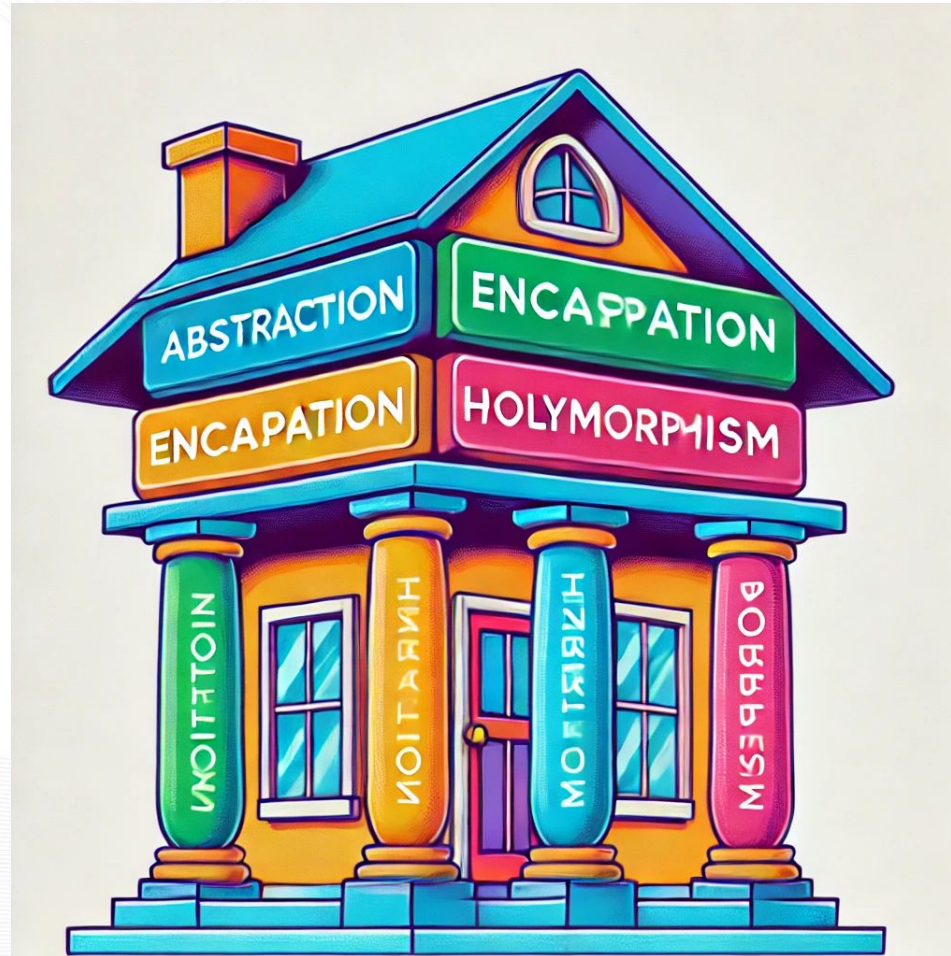
Por ahora, vamos a trabajarlo así:

05-objetos >  01-intro.js > ...

```
1  let user = {  
2      email: "diegokld23@gmail.com",  
3      name: "Diego",  
4      direccion: {  
5          calle: "Av. Siempre Viva",  
6          numero: 123,  
7      },  
8      activo: true,  
9      recuperaClave: function () {  
10         console.log("Recuperando clave");  
11     },  
12 };
```


Creación de un objeto.

Poo



Dinamismo



En Js, los objetos son dinámicos, es decir que puedes llamar en cualquier momento una propiedad o simplemente asignarle un valor diferente.

```
1  const user = {  
2      id: 1,  
3      name: 'Leanne Graham',  
4  };  
5  //Forma de llamar un mostrar un atributo del objeto  
6  console.log("El id de user es: ", user.id);
```


Dinamismo



En js, los objetos son dinámicos, es decir que puedes llamar en cualquier momento una propiedad o simplemente asignarle un valor diferente.

```
05-objetos > JS 02-dinamismo.js > ...
1  const user = {
2      id: 1,
3      name: 'Leanne Graham',
4  };
5  //Forma de llamar un mostrar un atributo del objeto
6  console.log("El id de user es: ", user.id);
7  //Forma de cambiar el valor de un atributo del objeto
8  user.id = 3;
9  console.log("El id de user es: ", user.id);|
```

Dinamismo



Otra forma de usar un objeto, es añadiendo atributos que no existen así:

05-objetos > JS 02-dinamismo.js > ...

```
1  const user = {  
2    |    id: 1,  
3  };  
4  
5  user.name = 'Pablo';  
6  user.guardar = function(){  
7    |    console.log('Guardando', user.name);  
8  }  
9  
10 user.guardar();
```


En la pantalla anterior apareció un nuevo concepto que recibe el nombre de funciones anónimas.

Las funciones anónimas en JavaScript son funciones que no tienen un nombre definido. Se utilizan comúnmente como funciones de callback o en expresiones de función. Aquí hay algunos puntos clave sobre las funciones anónimas:

Características de las Funciones Anónimas:

1.Sin Nombre:

1. No tienen un identificador o nombre. Se definen en línea y se pueden asignar a una variable o pasar como argumento.

2.Uso Común:

1. Se utilizan frecuentemente como callbacks, en eventos, o en expresiones de función.

Dinamismo



Siguiendo con el código, hay otro inconveniente, y es que declaramos una constante a la cual le estamos agregando contenido, en pocas palabras ¿estamos modificando una constante?.

Pues resulta que como tal la constante no se está modificando, lo que estamos modificando es el contenido de un objeto, ahora si intentamos modificar como tal la constante, ahí si js, va a llorar.

```
1  const user = {  
2    id: 1,  
3  };  
4  
5  user = "Hola";
```

```
✖ ▶ Uncaught  
  TypeError: Assignment to constant variable.  
    at 02-dinamismo.js:5:6
```

Dinamismo



Ahora, si lo que queremos es hacer que NO se modifique ni el objeto ni sus propiedades, debemos usar la palabra reservada `Object.freeze({})`.

```
const user1 = Object.freeze({  
  id: 1,  
  name: 'Pablo',  
});
```

```
const user1 = Object.freeze({  
  id: 1,  
  name: 'Pablo',  
});  
  
user1.name = 'Juan';  
  
console.log("El nuevo nombre es: ", user1.name);
```

```
El nuevo nombre es: Pablo  
> |
```


Dinamismo



También existe la posibilidad de que no podamos agregar propiedades, pero SI modificar las existentes.

```
const user1 = Object.seal({  
  id: 1,  
  name: 'Pablo',  
});  
  
user1.name = 'Juan';  
user1.direccion = 'Calle 123';  
  
console.log("El nuevo nombre es: ", user1.name);  
console.log("La dirección es: ", user1.direccion);
```

```
El nuevo nombre es: Juan  
La dirección es: undefined
```


Factory functions



Pensemos en un escenario donde tengamos que crear muchos objetos similares:

```
05-objetos > JS 03-factory.js > ...
1  let user = {
2    id: 1,
3    email: "diegokld23@gmail.con",
4    name: "Diego",
5    activo: true,
6    recuperaClave: function(){
7      console.log("Recuperando clave");
8    }
9  };
10 let user1 = {
11   id: 2,
12   email: "mariokld23@gmail.con",
13   name: "Mario",
14   activo: false,
15   recuperaClave: function(){
16     console.log("Recuperando clave");
17   }
18 };
```

Factory functions



En este punto el código se vuelve monótono y repetitivo, lo cual indica que algo va mal, así que empecemos a crear una función que nos ayude:

```
05-objetos > JS 03-factory.js > crearUsuario
1  function crearUsuario(name, email){
2      return {
3          id: 1,
4          email: email,
5          name: name,
6          activo: true,
7          recuperaClave: function(){
8              console.log("Recuperando clave");
9          }
10     };
11 }
```


Factory functions



Uno de los beneficios de js, es que si el nombre del parámetro es igual al de la propiedad, podemos eliminarlo y simplemente dejar el nombre del parámetro.

```
1  function crearUsuario(name, email){
2      return {
3          id: 1,
4          email,
5          ...name,
6          activo: true,
7          recuperaClave: function(){
8              console.log("Recuperando clave");
9          }
10     };
11 }
```

Factory functions



Uno de los beneficios de js, es que si el nombre del parámetro es igual al de la propiedad, podemos eliminarlo y simplemente dejar el nombre del parámetro.

```
1  function crearUsuario(name, email){
2      return {
3          id: 1,
4          email,
5          ...name,
6          activo: true,
7          recuperaClave: function(){
8              console.log("Recuperando clave");
9          }
10     };
11 }
```


Factory functions



Finalmente vamos a usar la factory function que acabamos de crear, para darle vida a varios usuarios.

```
const usuario = crearUsuario("Juan", "juan@gmail.com");  
const usuario2 = crearUsuario("Pepe", "pepe@gmail.com");  
const usuario3 = crearUsuario("Carlos", "carlos@gmail.com");
```

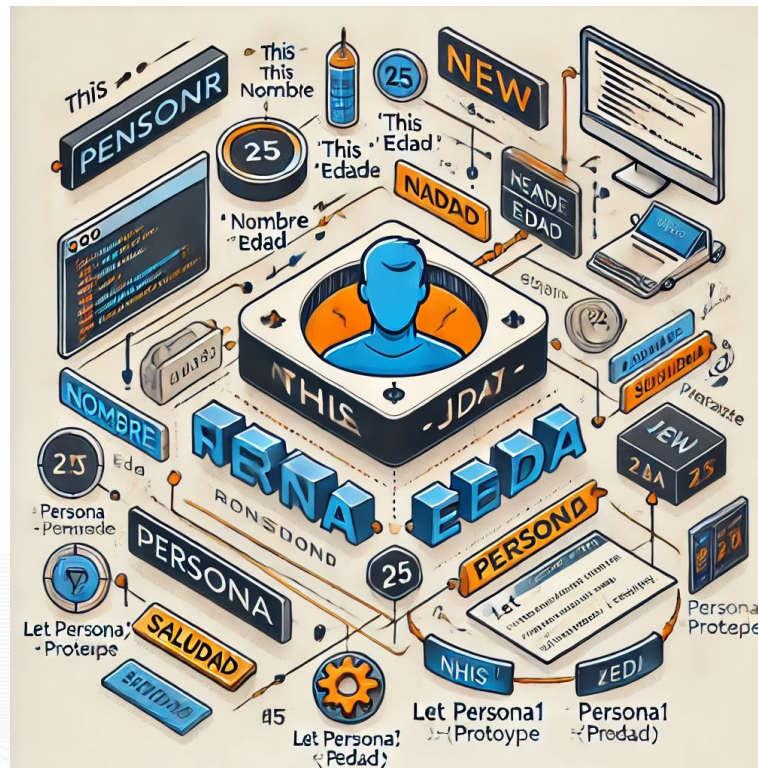
Y si imprimimos en la consola:

```
▶ {id: 1, email: 'juan@gmail.com', name: 'Juan', activo: true, recuperaClave: f}  
▶ {id: 1, email: 'pepe@gmail.com', name: 'Pepe', activo: true, recuperaClave: f}  
▶ {id: 1, email: 'carlos@gmail.com', name: 'Carlos', activo: true, recuperaClave: f}
```

Constructor function



Una función constructora en JavaScript es una función especial utilizada para crear e inicializar objetos. Las funciones constructoras actúan como plantillas para crear múltiples objetos similares, cada uno con sus propias propiedades y métodos. Estas funciones se utilizan junto con la palabra clave `new` para instanciar nuevos objetos.



Constructor function



Empecemos a crear dicha función. Lo primero que debemos saber es que usan UpperCamelCase;

```
1 function Usuario(){
2     //en caso de crear un propiedad con valor numerico
3     this.id = 1;
4     //en caso de crear una funcion
5     this.recuperarClave = function(){
6         console.log("Recuperando clave");
7     }
8 }
```

Bien, de esa forma creamos constructor? crear un usuario del objeto

```
let usuario = new Usuario();
console.log(usuario);
```

Para esto vamos a usar la palabra `new` otras palabras que vamos a crear un objeto de la función `Usuario`

```
Usuario {id: 1, recuperarClave: f}
```

Constructor function



Ahora, algo que debemos dejar muy claro, es que la función que se crea dentro de un constructor, ya no se va a llamar función, ahora se va a llamar **método** ya que estos son una función que fue asignada a una propiedad.

Por otro, la palabra `new`, lo que hace es crear una instancia vacía, o como se conoce comúnmente: prototipo. Este objeto vacío se le asigna a la palabra reservada `this`, por lo tanto, `this`, será ahora un objeto vacío y finalmente, si es que la función de usuario no retorna nada, lo que hace, es que de manera automática retorna `this`, es decir, un objeto vacío.

Atajos constructores



En JavaScript, los atajos constructores son una forma de definir métodos de forma concisa dentro de un objeto o una clase, introducida con la sintaxis de ES6. Esto permite definir métodos directamente dentro de los literales de objeto o en las clases sin necesidad de utilizar la palabra clave function.

```
05-objetos > JS 05-atajos.js > ...
1  let obj = {}
2  let obj2 = new Object();
3
4  /**
5   * new Array(); // []
6   * new String(); // ''
7   * new Number(); // 0
8   * new Boolean(); // false true
9   * new Date(); // Fecha actual
10  */
11
12  function Usuario(){
13      this.name = "Diego Calderon";
14  }
15
16  let user = new Usuario();
17  console.log(user.constructor);
```

```
f Usuario(){
    this.name = "Diego Calderon";
}
```

Funciones



Hasta ahora, hemos visto que las funciones se crean y trabajan como objetos, pero siempre sin recibir parámetros, esto puede cambiar, lo que significaría que el constructor estaría esperando siempre un parámetro para inicializar.

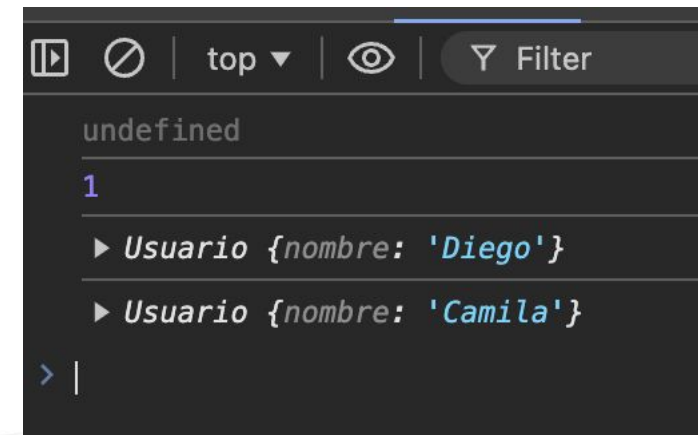
```
05-objetos > JS 06-funciones.js > ...
1  function Usuario(nombre){
2      |      this.nombre = nombre;
3  }
4
5  console.log(Usuario.nombre);
6  console.log(Usuario.length);
7
8
9  const U = new Usuario("Diego");
10 console.log(U.nombre);|
```


Funciones



También existe el caso donde podemos pasar funciones a otras funciones como argumento:

```
05-objetos > JS 06-funciones.js Cerrar (⌘W)
1  function Usuario(nombre){
2      this.nombre = nombre;
3  }
4
5  console.log(Usuario.nombre);
6  console.log(Usuario.length);
7
8
9  const U = new Usuario("Diego");
10 console.log(U);
11
12 function of(Fn, arg){
13     return new Fn(arg);
14 }
15
16 const user1 = of(Usuario, "Camila");
17 console.log(user1);
```

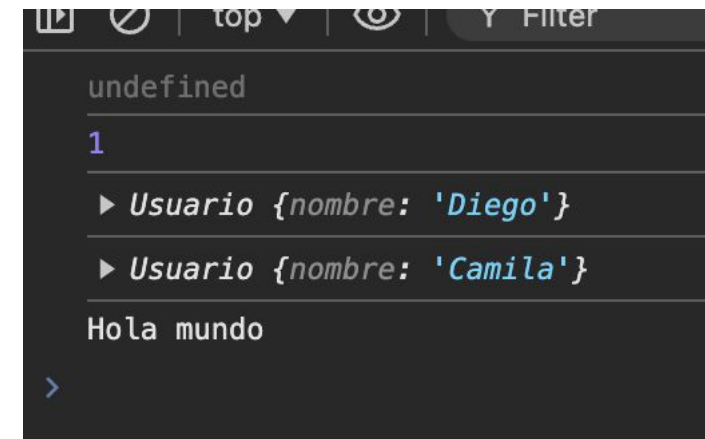


Funciones



Además, también las podemos retornar dentro de otras funciones.

```
19 function returned(){
20     return function(){
21         console.log("Hola mundo");
22     };
23 }
24
25 let saludo = returned();
26
27 saludo();
```



Funciones



Si bien la forma anterior de crear y escribir el código funciona, no es la mejor en js, para esto existen un par de palabras reservadas llamadas `call` y `apply` las cuales nos ayudan a extender los objetos.

05-objetos > JS 07-function.js > ...

```
1  function Punto(x, y) {  
2    this.x = x;  
3    this.y = y;  
4    this.dibujar = function () {  
5      console.log("Dibujando punto en x: " + this.x + ", y: " + this.y);  
6    };  
7  }  
8  
9  let punto = { z: 7 };  
10 Punto.call(punto, 5, 6);  
11  
12 console.log(punto);  
13
```

```
▶ {z: 7, x: 5, y: 6, dibujar: f}
```

Funciones



Si bien la forma anterior de crear y escribir el código funciona, no es la mejor en js, para esto existen un par de palabras reservadas llamadas `call` y `apply` las cuales nos ayudan a extender los objetos.

```
objetos > js 07-function.js > ...  
1  function Punto(x, y) {  
2    this.x = x;  
3    this.y = y;  
4    this.dibujar = function () {  
5      console.log("Dibujando punto en x: " + this.x + ", y: " + this.y);  
6    };  
7  }  
8  
9  let punto = { z: 7 };  
10 Punto.apply(punto, [5, 6]);  
11  
12 console.log(punto);
```

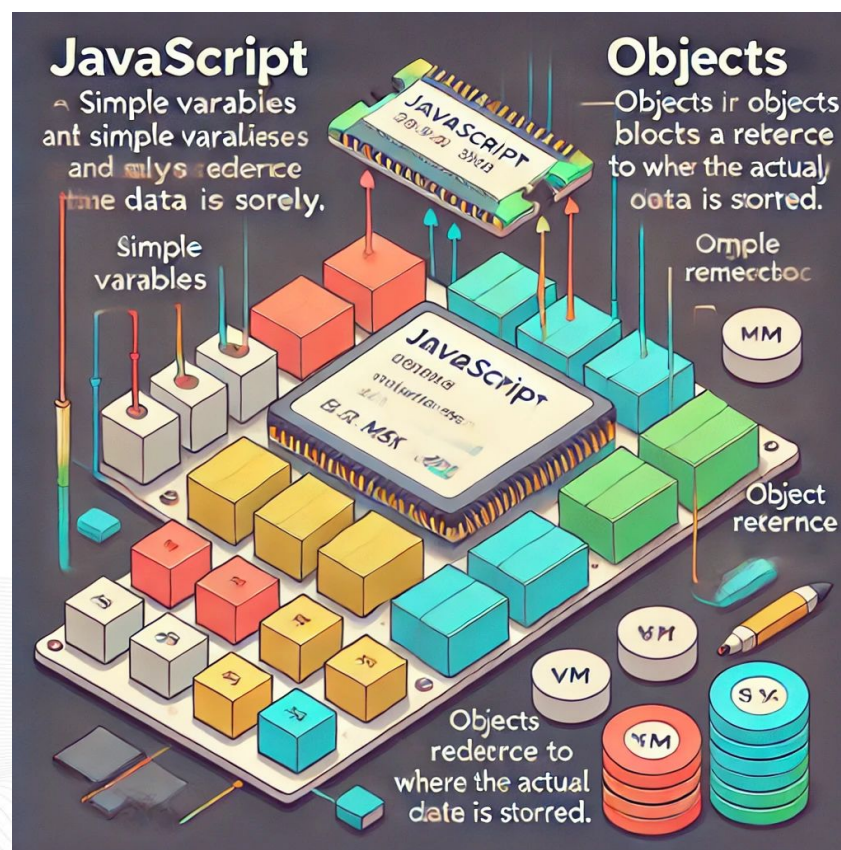
A screenshot of a browser's developer console showing the output of the JavaScript code. The output is an object: `{z: 7, x: 5, y: 6, dibujar: f}`. The `z` property is highlighted in blue, `x` in purple, `y` in blue, and `dibujar` in blue. The `f` represents a function.

```
► {z: 7, x: 5, y: 6, dibujar: f}
```


Valor y referencia.



Cuando almacenamos variables en la memoria RAM, JavaScript toma un bloque de memoria y le asigna el valor de la variable. Sin embargo, si definimos un objeto, lo que se guarda en la memoria no es el objeto en sí, sino la dirección donde se encuentra almacenado el objeto. De hecho, ese bloque u objeto puede estar disperso al final de la memoria RAM.



Listar de propiedades



JavaScript, naturalmente es un lenguaje interactivo, es por esto que las propiedades que creemos se pueden ir agregando o quitando todo el tiempo. Veamos como listar las propiedades de un objeto que fueron creadas de manera dinámica.

```
js > 09-listProps.js > ...
const punto = {
  x: 10,
  y: 20,
  dibujar() { //metodo sin : function
    console.log('Dibujar');
  }
};

if('dibujar' in punto){
  punto.dibujar();
}
```

```
js > 09-listProps.js > ...
const punto = {
  x: 10,
  y: 20,
  dibujar() { //metodo sin : function
    console.log('Dibujar');
  }
};
delete punto.dibujar;
if('dibujar' in punto){
  punto.dibujar();
}
```


Listar de propiedades



Lo siguiente nos permitirá conocer las keys del objeto para luego trabajarlas:

```
// console.log(Object.keys(punto));  
for(let llave of Object.keys(punto)){  
  console.log(llave, punto[llave]);  
}
```

```
x 10  
y 20  
dibujar f dibujar() { //metodo sin : function  
        console.log('Dibujar');  
}
```

Listar de propiedades



La forma de como podemos acceder a los valores de las propiedades es la siguiente

```
for(let entry of Object.entries(punto)){  
    console.log(entry);  
}
```

A screenshot of a code editor's console output. The title bar shows icons for a file, a close button, and a dropdown menu set to "top". There is also an eye icon and a "Filter" button. The output is under the heading "Dibujar". It shows three entries, each preceded by a right-pointing triangle and "(2)".

```
Dibujar  
▶ (2) ['x', 10]  
▶ (2) ['y', 20]  
▶ (2) ['dibujar', f]  
> |
```


Listar de propiedades



Lo anterior si nos toca mantener código, ya que es una forma vieja de escribir, como deberíamos escribirlo es como ya lo vimos:

```
for (let llave in punto){  
    console.log(llave, punto[llave]);  
}
```

```
Dibujar  
x 10  
y 20  
dibujar f dibujar() {//metodo sin : function  
    console.log('Dibujar');  
}
```

Duplicar objetos



Veamos como clonar objetos, de manera que dos variables no se encuentren apuntando al mismo objeto necesariamente. Para esto vamos a ver un metodo del constructor Object, que se llama assign, el cual nos permite asignar las propiedades de la derecha al de la izquierda.

```
objetos > JS 10-duplicar.js > ...  
1  let punto = {  
2    x: 10,  
3    y: 20,  
4  };  
5  
6  //Asignando a punto, la propiedad z con valor 20  
7  Object.assign(punto, { z: 20})  
8  console.log(punto);
```

```
▶ {x: 10, y: 20, z: 20}
```


Duplicar objetos



Ahora, ¿cómo puedo usar assign para clonar objetos?.

Facil, asignando un objeto vacío, antes del primer parámetro y objeto que pasamos, sin embargo, vamos a necesitar una referencia, por lo que le daremos un valor de retorno en donde indicamos que es un clon

```
1  let punto = {  
2    x: 10,  
3    y: 20,  
4  };  
5  
6  // Clonando punto con el método assign  
7  let clonePunto = Object.assign({}, punto, { z: 20 });  
8  console.log(  
9    `punto: ${JSON.stringify(punto)}, clonePunto: ${JSON.stringify(clonePunto)}`  
10 );
```

```
punto: {"x":10,"y":20}, clonePunto: {"x":10,"y":20,"z":20}
```

Si nos fijamos, punto es el inicial, pero clonePunto, tiene las propiedades de punto + la asignación que se hizo

Duplicar objetos



Ahora, que pasaría si al objeto literal “z:20” le paso una nueva propiedad que sea por ejemplo “x:1”

```
let punto = {  
  x: 10,  
  y: 20,  
};  
  
// Clonando punto con el método assign  
let clonePunto = Object.assign({}, punto, { z: 20, x:1 });  
console.log(  
  `punto: ${JSON.stringify(punto)}, clonePunto: ${JSON.stringify(clonePunto)}`  
);
```

```
punto: {"x":10,"y":20}, clonePunto: {"x":1,"y":20,"z":20}
```

Como se puede observar, el valor predominante es el de la derecha, y de esta forma se asigna siempre en el clon

Duplicar objetos



Recordemos que podemos clonar y asignar valores, pero en conclusión la forma de sacar una copia exacta, es pasando un objeto literal, junto del objeto que queremos clonar.

```
etos > Js 10-duplicar.js > ...  
1  let punto = {  
2    x: 10,  
3    y: 20,  
4  };  
5  
6  let copia = Object.assign({}, punto);  
7  console.log(copia, punto);  
8
```

```
▶ {x: 10, y: 20} ▶ {x: 10, y: 20}
```

Duplicar objetos



Esta forma que acabamos de ver es la forma más antigua, hoy en día ha cambiado. El operador de propagación (spread operator) en JavaScript se escribe utilizando tres puntos (...) Se utiliza para expandir elementos de un iterable (como un array o un objeto) en lugares donde se esperan múltiples elementos.

```
tos > .js 10-duplicar.js > ...  
  ▾ let punto = {  
    x: 10,  
    y: 20,  
  };  
  
  // Clonando punto y agregando una nueva propiedad usando el spread operator  
  let copia = { ...punto };  
  let clonePunto = { ...punto, z: 30 };  
  
  console.log(`punto: ${JSON.stringify(punto)}, copia: ${JSON.stringify(copia)}`);  
  console.log(`punto: ${JSON.stringify(punto)}, clonePunto: ${JSON.stringify(clonePunto)}`);
```

```
punto: {"x":10,"y":20}, copia: {"x":10,"y":20}
```

```
punto: {"x":10,"y":20}, clonePunto: {"x":10,"y":20,"z":30}
```


Público vs Privado



Cuando estamos usando la función constructora para poder crear objetos como por ejemplo usuario, va a existir la ocasión en donde queramos que algunos de sus métodos o propiedades no sean accesibles por todo el mundo.

En programación orientada a objetos (POO) en JavaScript, los términos "público" y "privado" se refieren al acceso y visibilidad de las propiedades y métodos dentro de una clase u objeto. Aquí te explico en detalle:

Público

Las propiedades y métodos públicos son accesibles desde cualquier parte del programa. En JavaScript, por defecto, todas las propiedades y métodos de una clase son públicos.

```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre; // Propiedad pública  
    this.edad = edad; // Propiedad pública  
  }  
  
  saludar() { // Método público  
    console.log(`Hola, mi nombre es ${this.nombre} y tengo ${this.edad} años.`);  
  }  
}  
  
let persona1 = new Persona("Carlos", 30);  
console.log(persona1.nombre); // Acceso público a la propiedad  
* persona1.saludar(); // Llamada pública al método
```

Público vs Privado



Privado

Las propiedades y métodos privados no son accesibles desde fuera de la clase. En JavaScript, las propiedades y métodos privados se pueden definir usando el prefijo #.

```
11-privadoPublico.js > ...
class Persona {
  #nombre; // Propiedad privada
  #edad; // Propiedad privada

  constructor(nombre, edad) {
    this.#nombre = nombre;
    this.#edad = edad;
  }

  #saludar() { // Método privado
    console.log(`Hola, mi nombre es ${this.#nombre} y tengo ${this.#edad} años.`);
  }

  mostrarSaludo() { // Método público que llama al método privado
    this.#saludar();
  }
}

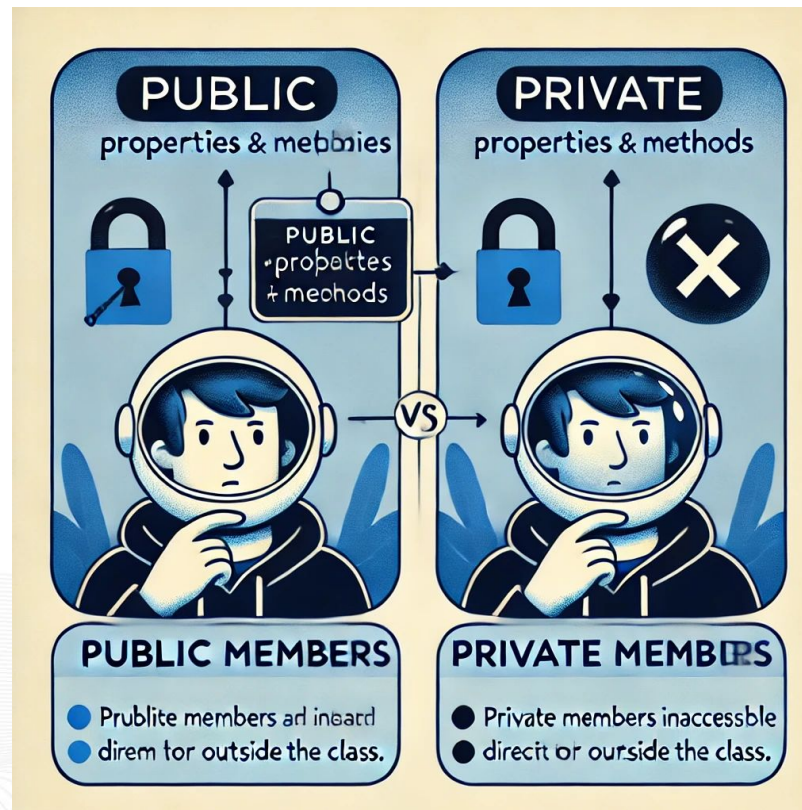
let persona1 = new Persona("Carlos", 30);
console.log(persona1.#nombre); // Error: propiedad privada no accesible
persona1.mostrarSaludo(); // Llamada a método público que a su vez llama al método privado
```

✖ Uncaught
SyntaxError: Private field '#nombre' must be declared in an enclosing class (at [11-privadoPublico.js:20:23](#))

Público vs Privado

Resumen:

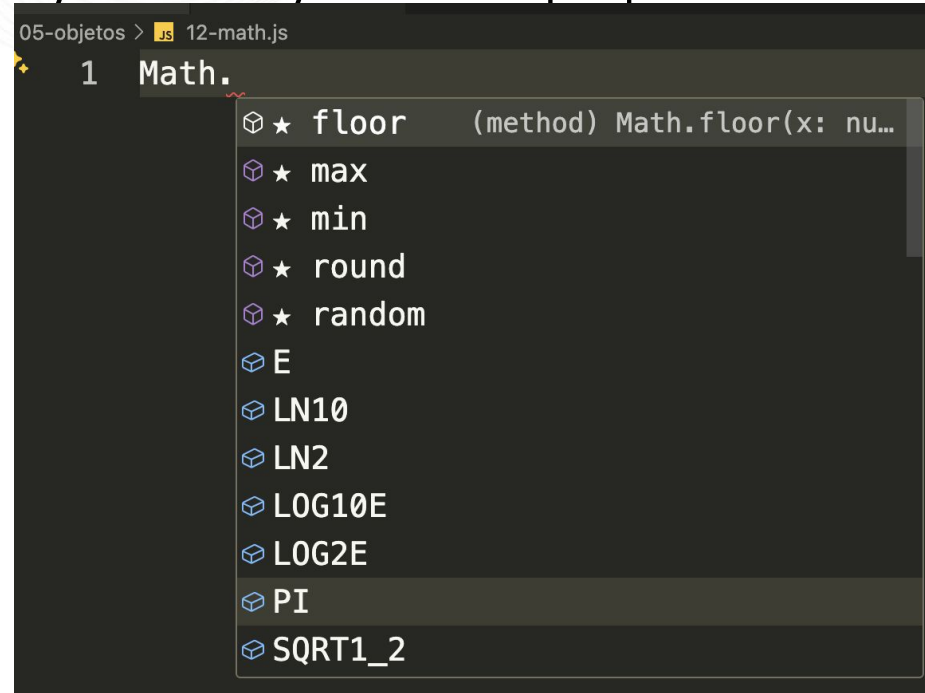
- **Propiedades y métodos públicos:** Son accesibles desde cualquier parte del código.
- **Propiedades y métodos privados:** Solo son accesibles desde dentro de la clase. En JavaScript moderno, se utiliza el prefijo # para definir propiedades y métodos privados.



Math



Dentro de js, hay muchos objetos y métodos ya escritos que podemos usar, uno de estos es “Math”



Si vemos hay muchos métodos que ya están listos para usar.

Math



Usemos el objeto Math

```
index.html  JS 12-math.js ×  
05-objetos > JS 12-math.js  
1 console.log(  
2     Math.PI,  
3     Math.abs(-5),  
4     Math.ceil(5.2),  
5     Math.floor(5.9),  
6     Math.round(5.5),  
7 );
```

3.141592653589793 5 6 5 6

Date



Al igual que Math, existe también, Date()

```
let ahora = new Date();
console.log(ahora);
let fechaEspecifica = new Date(2024, 7, 4, 14, 30, 0); // Año, Mes (0-11), Día, Hora, Minuto, Segundo
console.log(fechaEspecifica);
let desdeCadena = new Date("August 4, 2024 14:30:00");
console.log(desdeCadena);
let desdeTimestamp = new Date(1596522600000);
console.log(desdeTimestamp);
```

```
let fecha = new Date();
console.log(fecha.getFullYear()); // Año
console.log(fecha.getMonth()); // Mes (0-11)
console.log(fecha.getDate()); // Día del mes (1-31)
console.log(fecha.getHours()); // Hora
console.log(fecha.getMinutes()); // Minuto
console.log(fecha.getSeconds()); // Segundo
console.log(fecha.getMilliseconds()); // Milisegundo
console.log(fecha.getDay()); // Día de la semana (0-6)
```

Sun Aug 04 2024 14:30:00 GMT-0500 (hora estándar de Colombia)

Tue Aug 04 2020 01:30:00 GMT-0500 (hora estándar de Colombia)

2024

7

4

11

6

33

183

0

Date



En un ejemplo lo podemos usar así:

```
let fecha1 = new Date("2024-08-01");  
let fecha2 = new Date("2024-08-04");  
  
let diferenciaMilisegundos = fecha2 - fecha1;  
let diferenciaDias = diferenciaMilisegundos / (1000 * 60 * 60 * 24);  
  
console.log(`La diferencia es de ${diferenciaDias} días.`);
```

La diferencia es de 3 días.

Strings



El objeto String en JavaScript se utiliza para trabajar con cadenas de texto. JavaScript proporciona una variedad de métodos para manipular y trabajar con cadenas.

```
let cadena = "Hola, mundo!";  
let cadena1 = new String("Hola, mundo!");  
console.log(cadena);  
console.log(cadena1);
```

Hola, mundo!

► String {'Hola, mundo!'}

El objeto String y sus métodos proporcionan una amplia variedad de formas de trabajar con texto en JavaScript. Desde manipular y transformar cadenas, hasta buscar y extraer partes de ellas.

Strings



Un caso de uso:

```
let cadena = "Hola, mundo!";  
let cadenaInvertida = cadena.split("").reverse().join("");  
console.log(cadenaInvertida); // '!odnum ,aloH'
```

!odnum ,aloH



Caracteres de escape



Los caracteres de escape en JavaScript son secuencias de caracteres que no se interpretan de manera literal, sino que tienen un significado especial. Se utilizan dentro de las cadenas para representar caracteres que de otra manera serían difíciles de incluir directamente, como comillas, saltos de línea, tabulaciones, etc.

```
let cadena1 = 'I\'m learning JavaScript';  
console.log(cadena); // "I'm learning JavaScript"
```

```
let cita = "He said, \"Learning JavaScript is fun!\"\nAnd I agree."  
console.log(cita);  
// Output:  
// He said, "Learning JavaScript is fun!"  
// And I agree.
```


Caracteres de escape



Los caracteres de escape en JavaScript son secuencias de caracteres que no se interpretan de manera literal, sino que tienen un significado especial. Se utilizan dentro de las cadenas para representar caracteres que de otra manera serían difíciles de incluir directamente, como comillas, saltos de línea, tabulaciones, etc. La forma de usarlo es mediante `\${variable}`

```
let cadena1 = 'I\'m learning JavaScript';  
console.log(cadena); // "I'm learning JavaScript"
```

```
let cita = "He said, \"Learning JavaScript is fun!\"\nAnd I agree."  
console.log(cita);  
// Output:  
// He said, "Learning JavaScript is fun!"  
// And I agree.
```

Template strings



Los **template strings** (o plantillas de cadena) en JavaScript son una forma más flexible y potente de trabajar con cadenas de texto. Introducidos en ES6, los template strings permiten incluir expresiones embebidas, multilínea, y caracteres de escape de manera más sencilla. Se definen utilizando las comillas invertidas (backticks) ``.

```
//interpolacion
let nombre = "Carlos";
let edad = 30;
let mensaje = `Hola, mi nombre es ${nombre} y tengo ${edad} años.`;
console.log(mensaje); // "Hola, mi nombre es Carlos y tengo 30 años."
```

```
//multilinea
let multilinea = `Esta es una cadena
que abarca varias
líneas.`;
console.log(multilinea);
/* Salida:
Esta es una cadena
que abarca varias
líneas.
*/
```


Template strings



Ej.

```
//Expresiones complejas:  
let a = 10;  
let b = 20;  
let suma = `La suma de ${a} y ${b} es ${a + b}.`;   
console.log(suma); // "La suma de 10 y 20 es 30."
```

```
//uso de funciones  
function saludar(nombre) {  
    return `Hola, ${nombre}!`;   
}  
  
let mensaje = `${saludar("Carlos")}`;  
console.log(mensaje); // "Hola, Carlos!"
```

Ejercicios



1. Crear una función/clase que al instanciarla devuelva el nombre y el id(número aleatorio) de un usuario. Esta función debe llamarse Usuario y debe recibir un parámetro llamado name. La respuesta debe verse así: El el User es: {"id":0.9677623214119684,"name":"Pepe"} y el user1 es: {"id":0.21349024794256466,"name":"Juan"}
2. Haciendo uso de la factory function, retorne dentro de una función similar a la anterior las propiedades para que retorne un objeto.

El el User es: {"id":0.6963270730294406,"name":"Pepe"} y el user1 es: {"id":0.33946102598442374,"name":"Juan"}

3. Construir una función que me permita saber si sons objetos son similares. Cree una función llamada similares, la cual recibirá dos parámetros, obj1 y obj2 y determine si sus propiedades son similares. La respuesta debe ser un valor booleano dependiendo de los objetos que reciba, true, si son similares, false si no lo son.



GRACIAS

Línea de atención al ciudadano: 01 8000 910270
Línea de atención al empresario: 01 8000 910682



www.sena.edu.co